proxy_cache_challenge.sh

Revision: 2 **Last modified:** 2026-07-01T14:45:00Z **Authority:** Helix Constitution §11.4.27 (Challenges), §11.4.69 (sink-side positive evidence), §11.4.107 (forgeable-header discipline), §11.4.3 (honest SKIP), §11.4.68 (no fail-open)

Overview

Anti-bluff Challenge that proves the **live HTTP proxy** (default `http://localhost:53128`) caches. A **reliably-cacheable** plain-HTTP URL (default `http://ftp.debian.org/debian/README` — a Debian mirror README served with a cache-permitting `Cache-Control`) is fetched **twice** through the proxy; the **authoritative** proof is a Squid `TCP_*HIT` result code for that URL in the Squid `access.log`, asserted via `assert_cache_hit` from `tests/lib/evidence.sh`. An `X-Cache` header or an `Age` field alone is **forgeable** and is never accepted as the verdict — it is captured only as supplementary evidence. The `access.log` is read **inside the Squid container** by default (`podman exec proxy-squid`, READ-ONLY) because the rootless-container log is owned by a subuid (mode `0640`) and is typically **not** host-readable; a host-readable path is the fallback. The log length is snapshotted **before** the fetches and only the **appended** lines are inspected (a stale HIT from an earlier run can never satisfy the gate — mirrors `tests/security/proxy_acl_security.sh` S4). If **no** log source is reachable (container exec fails **and** the host log is unreadable), the challenge SKIPs with the closed-set reason `topology_unsupported` (§11.4.3); it never fakes a PASS.

Prerequisites

- The base proxy stack UP with the HTTP proxy on `53128` and the Squid container running (default name `proxy-squid`).
- `curl`, `bash`, `awk`, `grep`; `tests/lib/evidence.sh` present (sourced).
- `podman` on `PATH` for the authoritative container-log read. If `podman` is absent or the container is down, the script falls back to the host log path and, if that too is unreadable, honestly SKIPs.
- READ-ONLY: a plain proxy client + a `podman exec … wc -l/tail` (or host-file) read of the access.log — **never** stops/starts/reconfigures any container.

Usage

```
bash challenges/scripts/proxy_cache_challenge.sh
# override the cache url / container / host log path / evidence
        dir:
CACHE_URL=http://ftp.debian.org/debian/README \
SQUID_CONTAINER=proxy-squid \
SQUID_CONTAINER_LOG=/var/log/squid/access.log \
SQUID_ACCESS_LOG=/path/to/host/access.log \
CHALLENGE_EVIDENCE_DIR=/path/to/evidence \
  bash challenges/scripts/proxy_cache_challenge.sh
```

Environment: `HTTP_PROXY_URL` (default `http://localhost:53128`), `CACHE_URL` (default `http://ftp.debian.org/debian/README`), `SQUID_CONTAINER` (default `proxy-squid` — the authoritative log is read here), `SQUID_CONTAINER_LOG` (default `/var/log/squid/access.log`), `SQUID_ACCESS_LOG` (override the resolved **host fallback** path), `LOG_DIR` (default `./logs`; the compose file maps container `/var/log/squid` here), `CHALLENGE_EVIDENCE_DIR` (default `qa-results/challenges/<run-ts>`), `CURL_MAX_TIME` (default `20`).

Access-log resolution

Authoritative source (preferred): read inside the container via `podman exec $SQUID_CONTAINER … $SQUID_CONTAINER_LOG`. Host fallback: `SQUID_ACCESS_LOG` → `$LOG_DIR/access.log` (from env) → `.env` `LOG_DIR` → `./logs/access.log` (the `docker-compose.yml` default mount). Whichever source is used, its length is snapshotted **before** the double-fetch and only the appended lines are asserted against.

Exit codes

Code	Meaning
`0`	PASS — a Squid `TCP_*HIT` for the URL was found in the **appended** access.log lines this run produced (read via the container or host).
`1`	FAIL — proxy could not fetch the reachable URL, **or** the log showed **no** `TCP_*HIT` for a URL fetched twice (e.g. an uncacheable url — the legitimate negation of the gate).
`3`	SKIP — outage (URL unreachable) **or** no access.log source is reachable (container exec fails and host log unreadable — `topology_unsupported`, §11.4.3).

Anti-bluff notes

- The connectivity precondition uses `proxy_conn_verdict` so a site outage SKIPs while a proxy-broken-but-site-reachable-directly case FAILs.
- `Via:` / `Age:` / `X-Cache:` headers are logged as **supplementary** only (the origin's own Varnish `X-Cache: HIT` is meaningless for *Squid* caching); the verdict rests on the data-plane Squid `TCP_*HIT` result code, per §11.4.107.
- The log length is snapshotted **before** the fetches; only appended lines are asserted, so a stale HIT can never produce a false PASS.
- Negation is real: an **uncacheable** url (e.g. `http://example.com/`, `http://code.jquery.com/…`) is fetched with `200` but logs `TCP_MISS` twice ⇒ the challenge FAILs (exit 1), it does **not** rubber-stamp any double-fetch.
- No log source ⇒ honest `topology_unsupported` SKIP with the container name + host `ls -ln` ownership captured — never a fabricated cache PASS.

Outputs

`<evidence-dir>/cache/cache_evidence.txt` (double-fetch codes, `Via`/`Age`/`X-Cache` headers, connectivity verdict, the log-source + snapshot metadata, and either the `assert_cache_hit` verdict with the appended `TCP_*` result codes or the no-log-source SKIP evidence), `<evidence-dir>/cache/access_appended.log` (the appended access.log lines this run inspected), plus the two raw header dumps.

Related scripts

- `proxy_forward_http_challenge.sh`, `proxy_socks5_challenge.sh` — forwarding proofs.
- `run_proxy_challenges.sh` — the bank runner.
- `tests/lib/evidence.sh` — `assert_cache_hit` and the sourced helpers.

Last verified: 2026-07-01 (live runs vs Squid 6.13 `proxy-squid`, cache_dir aufs + cache_mem 512 MB): (1) default `http://ftp.debian.org/debian/README` → precondition PASS, authoritative `TCP_MEM_HIT/200` in the container access.log appended lines → **OVERALL=PASS (exit 0)**; (2) negation `CACHE_URL=http://example.com/` → `200/200` but `TCP_MISS/200` twice → **OVERALL=FAIL (exit 1)**; (3) bogus `SQUID_CONTAINER` + unreadable host log → **SKIP `topology_unsupported` (exit 3)**. `sh -n` + `bash -n` clean.